GenAI Declaration — COMP3011 Coursework 1

Student: Yusuf Demir **Module:** COMP3011 Web Services and Web Data
Assessment: Individual Web Services API Development Project
Date: March 2026

Tools Used

Tool	Interface	Role in workflow
Google Gemini	Web chat	Project ideation — helped me choose what to build. Then produced a full requirements analysis document for the chosen project.
ChatGPT (OpenAI)	Web chat	Prompt engineering — I gave it Gemini's requirements document and asked it to write a structured, detailed prompt I could give to Claude/Copilot to build the project.
GitHub Copilot (Claude Sonnet 4.6)	VS Code extension	Primary implementation assistant — received the ChatGPT-engineered prompt and built the entire application. Also used for debugging, architecture discussions, and documentation.

My GenAI Workflow — The Full Story

I want to be completely transparent about how I used AI in this project. I didn't just use one tool as an autocomplete — I used three different AI tools in a deliberate pipeline where each one had a specific job. Here is exactly what happened, step by step.

The Pipeline

```
[Me] → Gemini (ideation + requirements) → ChatGPT (prompt engineering) →
Copilot/Claude (implementation)
```

The reason I used multiple tools rather than just one is that I found each AI has different strengths. Gemini was good at brainstorming and structured analysis. ChatGPT was better at understanding what a code-generation AI would need in a prompt. And Copilot/Claude was the actual builder — it works directly in VS Code and can see my files, run commands, and iterate on code in real time.

Declaration of AI Use — Full Detailed Log

1. Project Ideation — Google Gemini

Stage: Before any code was written
What I did: I genuinely did not know what project to build when I started. The brief gave several example ideas — book recommendation API, transport delay tracking, nutrition API, housing market data — and I wasn't sure which one would be the best fit for what I wanted to learn and what would score well.

So I opened Google Gemini and just had a conversation. I described the coursework requirements (REST API, real data, analytics, multi-user features) and went through each of the example project ideas. I told

Gemini about my experience level, that I was comfortable with TypeScript and Node, and asked it to help me think through the pros and cons of each option.

After about 15 minutes of back-and-forth, Gemini helped me converge on the **housing market analytics API** idea. The key arguments that convinced me were:

- The HM Land Registry Price Paid dataset is enormous, publicly available, well-structured, and free — no scraping or API keys needed
- Housing affordability is a topic with genuine public interest, so the project would have real-world relevance
- The dataset naturally supports rich analytics: price trends over time, regional comparisons, property type breakdowns, affordability ratios
- The data volume (hundreds of thousands of records) would let me demonstrate real database performance work (indexing, aggregation)

What Gemini contributed: The conversation helped me make a decision I was struggling with. Gemini didn't write any code — it helped me think through the project choice systematically. Without this conversation, I probably would have spent much longer going back and forth on my own, and I might have picked something less interesting.

Representative exchange (paraphrased from memory):

> ***Me:*** *I'm stuck between a transport delay API and a housing market API. The transport one seems simpler but the housing one seems like it could be more impressive. What do you think?*
>
> ***Gemini:*** *Both are valid, but the housing market option has a significant advantage — the HM Land Registry Price Paid dataset is one of the most comprehensive public datasets in the UK. You'd have hundreds of thousands of real transactions to work with, which gives you genuine analytics to build. A transport API would likely rely on smaller samples or scraping. The housing data also naturally supports the kind of multi-endpoint analytics your coursework requires: trends, regional breakdowns, affordability metrics. In terms of demonstrating database skills, having half a million records to index and aggregate is much more compelling than a few thousand transport entries.*

---

2. Requirements Analysis — Google Gemini

Stage: Still before any code
What I did: Once I had decided on the housing market project, I stayed in the same Gemini conversation and asked it to help me produce a proper requirements document. I told Gemini the technology stack I wanted to use — Node.js, Express, TypeScript, PostgreSQL, Prisma ORM, Vue 3 with Pinia — and asked it to generate structured requirements covering:

I chose that stack deliberately because I regularly work with Vue and TypeScript. Since most implementation would be AI-assisted, staying with a familiar stack let me verify Copilot's code quality properly instead of just trusting generated output.

- What endpoints the API needs to expose (and what each one should return)
- What the database tables should look like (what fields from the Land Registry CSV map to which columns)
- Non-functional requirements: authentication, pagination, error handling, input validation, CORS
- Analytics requirements: what kind of insights should the market analytics endpoint compute
- Frontend requirements: what views/pages, what state management, what user flows

Gemini produced a detailed, well-organised requirements specification. It wasn't perfect — I reviewed it and added a few things myself (the saved listings comparison feature, and the specific market signal

classification logic), but the core structure came from Gemini.

What Gemini contributed: A structured requirements document that became the foundation for the entire project. This saved me significant time compared to writing requirements from scratch, and the result was more thorough than what I would have produced alone because Gemini thought of edge cases I hadn't considered (like idempotent data imports and session expiry cleanup).

---

3. Prompt Engineering — ChatGPT

Stage: Still pre-development
What I did: This is the step that makes my workflow different from most. Instead of giving Gemini's requirements document directly to a code-generation AI, I gave it to **ChatGPT** first with a very specific task: "Take this requirements document and write me the best possible prompt I can give to Claude/Copilot to build this project from scratch."

The reason I did this is that I had noticed code-generation AIs produce much better results when the prompt is structured in a specific way — with explicit file layout, type definitions, layering rules, and constraints stated upfront. A broad requirements document doesn't have that structure. ChatGPT's job was to bridge the gap: translate "what I want" into "exactly how to instruct an AI to build it."

ChatGPT produced a long, detailed prompt that specified:

- Exact file and folder structure (backend/src/controllers/, backend/src/services/, etc.)
- TypeScript interface definitions for all API request/response types
- Layering rules: controllers should only handle HTTP concerns, services should contain all business logic, Prisma should only be called from services
- Security requirements stated explicitly: bcrypt with 12 rounds, crypto.randomBytes(48) for tokens, 7-day session expiry
- Error handling conventions: consistent `{ success, data/error }` envelope, specific HTTP status codes for each error type
- Test expectations: integration tests with Vitest + Supertest, test registration, login, CRUD, and filter combinations

What ChatGPT contributed: A precisely structured build prompt optimised for a code-generation AI. This was the most "meta" use of AI in the project — using one AI to write instructions for another AI. The quality of the Copilot output was noticeably better with the ChatGPT-engineered prompt than what I got when I tried giving raw requirements directly to Copilot in early experiments.

Representative exchange (paraphrased):

> ***Me:*** *Here is a requirements document for a UK housing market analytics API. I want you to write me a detailed, structured prompt that I can give to Claude/Copilot in VS Code to build this entire project. Make the prompt as precise as possible — specify file structure, types, layering conventions, security details, everything.*
>
> ***ChatGPT:*** *I'll structure this as a comprehensive build instruction. Key things to make explicit for a code-generation AI: (1) specify the exact directory layout so it doesn't improvise folder names, (2) define TypeScript interfaces upfront so types are consistent everywhere, (3) state architecture rules as hard constraints (e.g., "controllers MUST NOT call Prisma directly"), (4) list security requirements as non-negotiable specifications rather than suggestions. Here is the full build prompt...*

---

4. Full Implementation — GitHub Copilot (Claude)

Stage: Active development
What I did: I gave the ChatGPT-engineered prompt to GitHub Copilot in VS Code. Copilot (running Claude) then built the project iteratively. The process was not fully automatic — it was a back-and-forth conversation over many hours:

- **Phase 1 — Backend skeleton**: Copilot created the Prisma schema, then the service layer, controllers, routes, and middleware. Each component was generated in stages, and I reviewed the code after each stage before moving on.
- **Phase 2 — Analytics logic**: The trends and market insights endpoints were the most complex. Copilot wrote the initial implementations, but I had Copilot iterate on the market signal classification logic (adjusting the growth thresholds) and the affordability ratio formula.
- **Phase 3 — Auth system**: Copilot initially suggested JWT-based auth. I pushed back and asked it to implement database-backed sessions instead (see architecture refinement below).
- **Phase 4 — Frontend**: Copilot built the Vue 3 frontend — router, stores, views, and components — from the same prompt's frontend specifications.
- **Phase 5 — Tests**: Copilot generated the Vitest test suites, then I ran them and fed failures back for debugging.

My active involvement: I was not passively accepting code. I reviewed every file, questioned design choices, requested changes, and ran the application at each stage to verify it worked. When Copilot produced code I didn't understand, I asked it to explain the logic before accepting it. When tests failed, I shared the error output and discussed fixes.

What Copilot contributed: The actual code implementation. The project's codebase was built by Copilot from the structured prompt, with my continuous review and direction.

---

5. Architecture Refinement — Copilot

Stage: During implementation
What I did: Several times during the build, I stopped and had design discussions with Copilot:

- **JWT vs DB sessions**: Copilot's initial auth implementation used JWTs. I asked "what happens if a user wants to log out — can you actually invalidate a JWT?" This led to a discussion where Copilot explained that JWTs are stateless and can't be revoked without a blocklist. I decided I wanted instant logout, so I asked Copilot to rewrite the auth system to use database-backed sessions with a `sessions` table. Copilot did this and added the indexed `token` column for fast lookups.
- **Median vs mean price**: When building the insights endpoint, I asked whether average price was the right metric for housing data. Copilot pointed out that house prices are heavily right-skewed (a few very expensive properties pull the average up), so a median is more representative. We added `medianPrice` to the insights response.

---

6. Debugging Deployment — Copilot

Stage: Production deployment
What happened: After deploying the backend to Railway and the frontend to Vercel, I hit CORS preflight failures. The browser was sending OPTIONS requests that the backend rejected because the Vercel deployment URL wasn't in the allowed origins. I initially tried adding the specific Vercel URL to `CORS_ORIGIN`, but then realised Vercel generates a new unique URL for every deployment.

I described the problem to Copilot, including the browser console errors and Railway deploy logs. Copilot diagnosed the issue: Vercel preview deployments use URLs like `https://<hash>.vercel.app`, and

suggested implementing a regex check in the CORS origin callback: `^https://[a-z0-9-]+\.vercel\.app$` . This accepts any Vercel subdomain while still rejecting other origins.

Beyond CORS, I also struggled with Railway build failures (getting the `railway.json` build and start commands right), database URL confusion (internal vs external URLs), and slow data imports over the external database connection. Copilot helped me reason through each of these, but ultimately I had to experiment with configuration changes and redeploy multiple times to get everything working.

---

7. Test Coverage Review — Copilot

Stage: Testing phase
What I did: After writing the initial test suites, I asked Copilot to review them for coverage gaps. Copilot identified:

- Missing auth guard tests on saved listing routes (verifying that requests without tokens get 401)
- Missing 409 Conflict test for duplicate saved listings
- Missing combined-filter tests on the properties endpoint (filtering by both price range and property type simultaneously)

I then added all three: auth guard coverage, duplicate-saved-listing conflict coverage (`409`), and combined-filter coverage on the properties endpoint.

---

8. Documentation & Report — Copilot

Stage: Final documentation
What I did: I used Copilot to help structure this technical report, the GenAI declaration, and the presentation slides. The content, analysis, and design decision reasoning are all mine — I explained my thought process and Copilot helped organise it into clear sections with professional phrasing. Copilot also generated the Python script (`generate_presentation.py`) that programmatically builds the .pptx slide deck.

Reflective Analysis — What I Learned About Working With AI

What worked well

The multi-model pipeline was genuinely effective. Using Gemini for ideation gave me a better project choice than I would have made alone. The ChatGPT prompt engineering step was the most valuable — the quality difference between a raw requirements prompt and a specifically-engineered build prompt was significant. Code generated from the engineered prompt had consistent architecture, naming conventions, and error handling from the start.

What I would do differently

Looking back, I relied on AI too heavily for the initial implementation and not enough for testing and validation. While I reviewed all generated code, I wish I had written more of the core business logic myself — particularly the market insights aggregation pipeline. That would have given me a deeper understanding of the data manipulation and aggregation challenges. I also would have written more tests myself to ensure I had a strong grasp of the expected behavior and edge cases.

I also learned that deployment is the area where AI is least helpful. Copilot could help me reason about CORS configuration and suggest fixes, but it couldn't actually see my Railway deploy logs or Vercel build output in real time. The deployment debugging was mostly manual trial and error — change a config, push, wait for the deploy, check if it worked, repeat. This was the most frustrating and time-consuming part of the project.

Level of AI Use

The majority of the code in this project was generated by AI (Copilot/Claude), from a prompt that was itself written by AI (ChatGPT), based on requirements that were produced by AI (Gemini). My role was as the director — I chose the project, decided the stack, reviewed all code, made architectural decisions (like sessions over JWTs), debugged deployment issues, ran tests, and wrote the analytical content in this report. I did not write most of the code line-by-line myself, but I understand all of it and can explain it.

Self-Assessment of GenAI Usage Level

Based on the grade band rubric:

> *** Very High level use of GenAI to aid creative thinking and solution exploration.**

The multi-model pipeline (Gemini → ChatGPT → Copilot/Claude) represents a sophisticated and intentional approach to AI-assisted development. Each tool was selected for a specific strength:

- Gemini: brainstorming and structured analysis
- ChatGPT: meta-level prompt engineering
- Copilot/Claude: real-time code generation within the IDE

This is not a case of using one tool as a simple code autocomplete. The pipeline required me to understand what each AI tool is good at, orchestrate the handoff between tools, and maintain quality control throughout. I was actively involved at every stage: making the project choice, reviewing requirements, evaluating prompt quality, reviewing generated code, challenging architectural decisions, debugging deployment, and running tests.

Honesty Statement

All functionality, design decisions, and analysis described in the technical report and demonstrated in the oral examination are work I directed and reviewed, using AI tools as declared above. I reviewed all AI-generated code before acceptance and can explain every component of the system. All AI tool usage has been fully declared in this document. I have not omitted or hidden any use of AI.

This document was also generated with the assistance of AI tools, primarily ChatGPT for structuring and phrasing, based on my detailed explanations and analysis. I have reviewed and approved all content in this declaration. All of the markdowns was constructed using ai and i have reviewed and approved all content in the markdowns.

This declaration was prepared in accordance with the COMP3011 GenAI policy (Green Light Assessment).